

Unidad 2

Programación de hilos

Hilos en Java

Programación de servicios y procesos

Curso 2025/2026

Contenido

- ▶ Clase Thread
- ▶ Interfaz Runnable
- ▶ Métodos y atributos de la clase Thread
 - ▶ `join()`

Clase Thread

- ▶ Existe una clase en Java llamada Thread, que se puede usar para crear hilos heredando de ella e implementando (sobreescribiendo) su método “run”

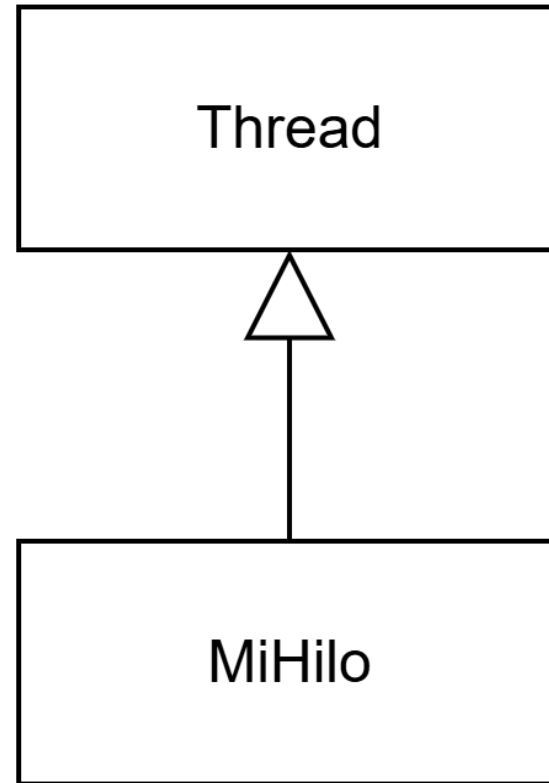
```
public class MiHilo extends Thread {  
    ... // atributos, constructores, métodos  
  
    @Override  
    public void run(){  
        // Código a ejecutar por el hilo  
    }  
}
```

Clase Thread

- ▶ Para lanzar el hilo, simplemente se llama al método start()
 - ▶ Esto ejecuta lo que se haya definido en el método run().

```
Thread t = new MiHilo();  
t.start();
```

Relación entre clases



Supuesto

- ▶ Supongamos tenemos una clase Ratón que incluye el método comer
- ▶ Cada ratón tarda en comer el tiempo que se indique en el constructor

Supuesto - clase Raton

```
public class Raton {  
    private String nombre;  
    private int tiempoAlimentacion;  
  
    public Raton(String nombre, int tiempoAlimentacion) {  
        super();  
        this.nombre = nombre;  
        this.tiempoAlimentacion = tiempoAlimentacion;  
    }  
  
    public void comer() {  
        try {  
            System.out.printf("El raton %s ha comenzado a alimentarse%n", nombre);  
            Thread.sleep(tiempoAlimentacion * 1000);  
            System.out.printf("El raton %s ha terminado de alimentarse%n", nombre);  
        } catch (InterruptedException e){  
            e.printStackTrace();  
        }  
    }  
}
```

Supuesto - clase principal

```
public class Main {  
    public static void main(String[] args) {  
        Raton fievel = new Raton("Fievel", 4);  
        Raton jerry = new Raton("Jerry", 5);  
        Raton pinky = new Raton("Pinky", 3);  
        Raton mickey = new Raton("Mickey", 6);  
        fievel.comer();  
        jerry.comer();  
        pinky.comer();  
        mickey.comer();  
    }  
}
```


Ejercicio 0

- ▶ Probar qué pasa al ejecutar la clase principal

Ejercicio 1

- ▶ ¿Cómo se podría hacer para que fueran hilos en paralelo usando herencia de la clase Thread?

Ejercicio 2

- ▶ ¿Qué pasa si cambiamos las llamadas a los métodos `start()` por métodos `run()`?

Interfaz runnable

- ▶ Otra posibilidad para crear hilos es definir una clase que implemente la interfaz Runnable e incorpore el código a ejecutar en su método “run”.

```
public class MiTarea implements Runnable {  
    ... // atributos, constructores, métodos  
    @Override  
    public void run() {  
        // Código a ejecutar por el hilo  
    }  
}
```

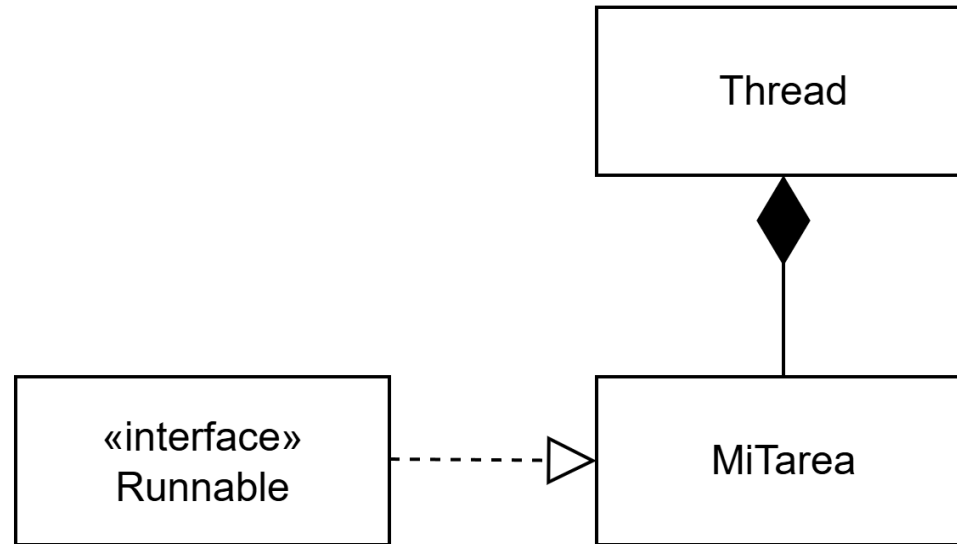
Interfaz runnable

- ▶ Si hemos definido el hilo implementado la interfaz Runnable, entonces podemos crear y ejecutar un hilo simplemente creando una nueva instancia de Thread con el objeto Runnable como parámetro

```
// Clase que implementa Runnable
MiTarea tarea = new MiTarea()
Thread t = new Thread(tarea);
t.start();
```

```
// Alternativa más compacta
Thread t = new Thread(new MiTarea());
t.start();
```

Relación entre clases



Ejercicio 3

- ▶ Repetir el ejercicio 1 con extensión de Runnable

En resumidas cuentas...

- ▶ Dada una clase que queramos convertir en un hilo:
- ▶ **PASO 1:** Hacer que herede de Thread o implemente Runnable
- ▶ **PASO 2:** Definir su método run()
- ▶ **PASO 3:** En el principal, después de crear un objeto de esa clase:
 - ▶ Si el objeto es un thread
 - ▶ Llamamos a su método start()
 - ▶ Si el objeto es un runnable
 - ▶ Creamos un thread con el objeto como parámetro
 - ▶ Llamamos a su método start()

¿Heredar de Thread o implementar Runnable?

- ▶ Si heredas de la clase Thread, no podrás heredar de ninguna otra clase (recuerda que Java no soporta la herencia múltiple).
- ▶ Si nuestra clase hereda de otra, es necesario implementarse la interfaz Runnable.
 - ▶ Es posible implementar múltiples interfaces, pero solo puedes heredar de una clase.
- ▶ Por tanto:
 - ▶ Aplicaciones sencillas en las que la clase se crea desde cero → herencia de Thread
 - ▶ Aplicaciones con jerarquía de clase → implementación de Runnable

Manera más compacta de definir un hilo

- ▶ Un Runnable solo implementa una función, es decir que es un **interfaz funcional**
- ▶ En Java los interfaces funcionales se pueden implementar con una **lambda**
 - ▶ Recordatorio: una lambda es una función anónima, se define con `() -> {}`
- ▶ Por tanto, un Runnable se puede definir en el sitio con una lambda, a la vez que se crea el Thread:

```
Thread hilo = new Thread(() -> {  
    System.out.println("Hilo ejecutándose");  
});  
hilo.start();
```

Clase Thread - método join()

- Sirve para esperar a que un Thread termine su ejecución

```
public static void main(String[] args) {  
    Thread t = new MiHilo();  
    t.start();  
    t.join();  
}
```

Clase Thread - método join()

- Puede generar una InterruptedException, por lo que debemos capturarla

```
public static void main(String[] args) {  
    Thread t = new MyThread();  
    t.start();  
    try {  
        t.join();  
    } catch (InterruptedException e) {  
        ...  
    }  
}
```

Ejercicio 4

- ▶ Repetir el ejercicio anterior añadiendo `System.exit(0)` después de lanzar los hilos ¿qué ocurre?

Ejercicio 5

- ▶ Antes del `System.exit(0)` ejecutar el método `join()` en cada ratón ¿qué ocurre?